

0.81. Ramasser une pièce : le P

Une pièce, la lettre P, en (15, 8). Quand le A arrive dessus, le score augmente de 1, et la pièce s'en va.

1. Ce qu'il faut comprendre

La pièce est une lettre, le P (`ALPHABET[15]`, la 16e lettre). On la pose une fois, au début, en (15, 8).

Ce qui est nouveau ici : comparer DEUX positions. Au 0.75, on comparait la position du A à une case fixe, (0, 0). Ici, on la compare à celle de la pièce, rangée dans `px` et `py` : `if (x == px && y == py)`. Le A est sur la pièce quand sa colonne **et** sa ligne sont celles de la pièce.

Ramassée : le score augmente (`score = score + 1;`), et la pièce part **hors de l'écran**, `px = 20;` (il n'y a pas de colonne 20). Ainsi le A, en restant sur la case, ne la ramasse pas une deuxième fois.

Elle disparaît de l'écran toute seule : en arrivant sur la case, `deplace_croix` pose le A **par-dessus** le P ; en repartant, il efface la case. Il ne reste rien.

Le score s'affiche en bas, après le mot SCORE.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1302 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	84 o	← la console, à l'allumage — la mise en route de la console
deplace_croix()	217 o	← main() — une fonction fournie par le langage
main()	94 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	704 o	← CopierTuiles — la police entière

"SCORE" 5 o ← texte() – un texte gravé

Lettres gravées : toutes (poser()) reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 2	26 o sur place, dans deplace_croix(), main() + AdresseCase (10 o)
bouton()	× 4	20 o sur place, dans deplace_croix() + LireManette (37 o)
effacer()	× 1	15 o sur place, dans deplace_croix() + AdresseCase (10 o), EffacerCases (12 o)
texte()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
nombre()	× 1	10 o sur place, dans main() + EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans deplace_croix()

Tes fonctions appelées :

deplace_croix()	× 1	19 o sur place, dans main()
-----------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----

#include <poser>           // pose une tuile sur une case du fond
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <texte>           // écrit un texte à l'écran
#include <deplace_croix>   // une tuile qui suit la croix, case par case
#include <nombre>          // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 5;           // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;           // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8
```

4. UNE VARIABLE — PX

```
uint8_t px = 15;         // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
```

5. UNE VARIABLE — PY

```
uint8_t py = 8;          // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.
```

6. UNE VARIABLE — SCORE

```
uint8_t score = 0;       // combien de pièces on a ramassées : 0 au départ
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- AVANT LA BOUCLE : ce qui ne se fait qu'une fois ----

    // La pièce est posée une seule fois, au départ.
    // poser(px, py, ALPHABET[15]);
    //      |   |   |
    //      |   |   +--- ALPHABET[15] : la 16e lettre (on compte à partir de 0), le P
    //      +---+----- sa place : (px, py) = (15, 8)
    poser(px, py, ALPHABET[15]);

    texte(0, 17, "SCORE");                // le mot SCORE, en bas à gauche (ligne 17)

    // ---- LA BOUCLE DU JEU : 60 fois par seconde ----
    while (true) {
        image();                          // 1. attend l'image suivante

        deplace_croix(x, y, ALPHABET[0], 250); // 2. le A suit la croix (le 0.71)

        // 3. Le A est-il sur la pièce ?
        //
        //   if (x == px && y == py)
        //       |           |
        //       |           +--- la LIGNE du A est-elle celle du P ?
        //       +----- la COLONNE du A est-elle celle du P ?
        //   Les DEUX doivent être vraies (&&) : le A est alors sur la case du P.
        //
        //   A en (14, 8) : x == px faux (14 ≠ 15)           → rien
        //   A en (15, 8) : x == px vrai ET y == py vrai     → ramassée !
        if (x == px && y == py) {
            score = score + 1;                // le score augmente de 1 : 0 → 1

            // La pièce part HORS de l'écran : la colonne 20 n'existe pas (0 à 19).
            // Sans cette ligne, le A resté sur la case la ramasserait encore à
            // l'image suivante, et le score monterait sans fin.
            px = 20;
        }

        // Le P a déjà disparu de l'écran : en arrivant, deplace_croix a posé le A
        // PAR-DESSUS ; en repartant, il a effacé la case. Il ne reste rien.

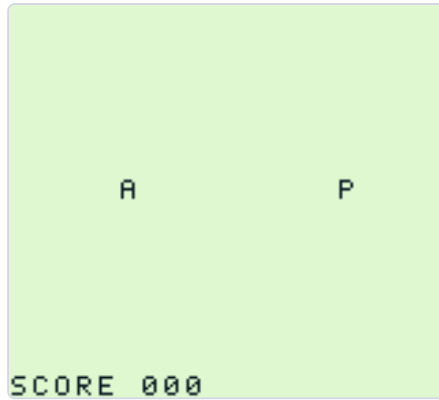
        nombre(6, 17, score);                // 4. le score, après le mot SCORE : 000, 001...
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A à gauche, un P à droite. Avec la flèche droite, le A ramasse le P : SCORE 001.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la pièce est un P, en (15, 8)
- ✓ ramassée : le score vaut 1
- ✓ une seule fois, même en passant dessus

Ensuite : **0.81.1. Ramasser une pièce : le P — elle réapparaît ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++